Indexation – Module Bases de Données

1. Problématique de l'indexation

L'indexation permet d'**optimiser l'exécution des requêtes** SQL en réduisant le nombre de lignes ou de blocs à parcourir. Elle améliore les performances en lecture mais introduit un coût supplémentaire en stockage et lors des opérations de mise à jour (INSERT, UPDATE, DELETE).

2. `EXPLAIN` : Présentation et usage

Définition

`EXPLAIN` est une commande SQL qui permet d'afficher le **plan d'exécution** d'une requête, c'est-à-dire la manière dont le SGBD accède aux données.

Objectifs

- Vérifier si une requête utilise un index
- Identifier **quel index** est utilisé
- Comparer les performances **avant / après indexation**
- Détecter les accès coûteux (full table scan)

Principe de fonctionnement

```
EXPLAIN SELECT ... ;
```

Le SGBD n'exécute pas la requête, mais **analyse** la stratégie d'accès :

- type d'accès aux données
- index utilisé
- nombre de lignes estimé
- opérations supplémentaires (tri, filtrage…)

Éléments importants du plan d'exécution

(les noms peuvent varier selon le SGBD)

- **Type d'accès**
  - `ALL` : parcours complet de la table ✗
  - `index` : parcours de l'index
  - `range` : recherche par intervalle
  - `ref` / `eq_ref` : recherche par clé indexée ✓
- **key** : index effectivement utilisé
- **rows** : nombre estimé de lignes lues
- **Extra**
  - `Using index` : index couvrant ✓
  - `Using where`
  - `Using filesort` : tri coûteux ✗

3. Types d’index

3 .1 Index Bitmap

Principe

- Un vecteur de bits par valeur distincte
- Chaque bit représente une ligne de la table
- Très efficace pour faible cardinalité

Usage

- Colonnes : sexe, statut, oui/non
- Requêtes avec conditions multiples
- Systèmes décisionnels

Limites

- Mauvaises performances en mise à jour
- Peu adapté à l’OLTP

Création (Oracle)

```
CREATE BITMAP INDEX idx_statut
ON Employe(statut);
```

Analyse avec EXPLAIN

```
EXPLAIN
SELECT * FROM Employe
WHERE statut = 'ACTIF';
```

Utilisation efficace du bitmap si cardinalité faible (nombre de valeur distinctes limité)

3.2 Index B-Tree

Principe

- Arbre équilibré
- Données ordonnées

Usage

- Index par défaut
- Égalité, intervalles, tri

Limites

- Moins performant que bitmap pour faible cardinalité

Création

```
CREATE INDEX idx_nom
ON Etudiant(nom);
```

Analyse avec EXPLAIN

```
EXPLAIN
SELECT * FROM Etudiant
WHERE nom = 'Ali';
```

➞Accès par index (`ref` ou équivalent)

3.3. Index Hash

Principe

- Fonction de hachage
- Accès direct par clé

Usage

- Recherche par égalité stricte (=)

Limites

- Pas d'intervalles
- Pas de `ORDER BY`

Création (selon SGBD)

```
CREATE INDEX idx_email_hash
ON Utilisateur USING HASH (email);
```

Analyse avec EXPLAIN

```
EXPLAIN
SELECT * FROM Utilisateur
WHERE email = 'test@mail.com';
```

➞Accès direct via hachage

4. Index avancés

4.1. Index Composite

Principe

- Index sur plusieurs colonnes
- Ordre des colonnes déterminant

Création

```
CREATE INDEX idx_nom_prenom
ON Etudiant(nom, prenom);
```

Analyse avec EXPLAIN

```
EXPLAIN
SELECT * FROM Etudiant
WHERE nom = 'Ali'
  AND prenom = 'Ahmed';
```

➞ Index composite utilisé

Non utilisé si la requête commence par `prenom` seul

4.2. Index Couvrant

Principe

- L'index contient toutes les colonnes demandées
- Pas d'accès à la table

Création

```
CREATE INDEX idx_nom_prenom
ON Etudiant(nom, prenom);
```

Analyse avec EXPLAIN

```
EXPLAIN
SELECT nom, prenom
FROM Etudiant
WHERE nom = 'Ali';
```

➞ `Using index` → index couvrant

5. Bonnes pratiques

- Toujours analyser les requêtes avec `EXPLAIN`
- Indexer les colonnes utilisées dans `WHERE`, `JOIN`, `ORDER BY`
- Éviter la sur-indexation
- Adapter le type d'index au contexte

6. Exemple : Gain de temps avec un index B-Tree

Table : `Etudiant` (10 000 lignes pour simuler un vrai scénario)

id	nom	prenom	age
1	Ali	Ahmed	20
2	Khan	Sara	22
…	…	…	…
10000	Li	Ming	23

6.1. Requête sans index

```
EXPLAIN SELECT * FROM Etudiant WHERE nom = 'Ali';
```

Plan d'exécution estimé :

id	table	type	key	rows	Extra
1	Etudiant	ALL	NULL	5000	Using where

- `type = ALL` → full table scan
- `rows = 5000` → le SGBD doit lire 5000 lignes en moyenne
- Temps d'exécution réel (mesuré avec `SELECT BENCHMARK` ou `EXPLAIN ANALYZE`) : **≈ 150 ms**

6.2. Création d'un index B-Tree sur `nom`

```
CREATE INDEX idx_nom ON Etudiant(nom);
```

6.3. Requête avec index

```
EXPLAIN SELECT * FROM Etudiant WHERE nom = 'Ali';
```

Plan d'exécution estimé :

id	table	type	key	rows	Extra
1	Etudiant	ref	idx_nom	50	Using where

- `type = ref` → l'index est utilisé
- `rows = 50` → seulement 50 lignes lues au lieu de 5000
- Temps d'exécution réel : **≈ 5 ms**

6.4. Gain de temps

- Sans index : lecture complète → 150 ms
- Avec index : lecture ciblée → 5 ms
- **Gain ≈ 30× plus rapide**

Conclusion :

- L’index B-Tree permet de localiser directement les lignes correspondant à `nom = 'Ali'` sans scanner toute la table
- Plus la table est grande, **plus l’impact de l’index est important**
- Idéal pour colonnes souvent filtrées dans les requêtes